SMU Lyle School of Engineering



Reinforcement Learning

Deep Reinforcement Learning

Based on An Introduction to Deep Reinforcement Learning**

+ Sutton/Barto* Sections 9.7 and 15.7

Michael Hahsler, SMU



Online Material

CC BY NC SA

*Sutton and Barto, Reinforcement Learning: An Introduction, 2nd edition, MIT Press, Cambridge, MA, 2018

**Vincent François-Lavet, Peter Henderson, Riashat Islam, Marc G. Bellemare and Joelle Pineau (2018), "An Introduction to Deep Reinforcement Learning", *Foundations and Trends in Machine Learning*: Vol. 11, No. 3-4. DOI: 10.1561/2200000071.

Topics of this Course

- Introduction to reinforcement learning
- Markov decision processes
- Part I: Tabular Methods
  - Dynamic programming
  - Monte Carlo methods
  - Temporal-difference learning
  - Multi-step bootstrapping
  - Planning and learning with tabular methods
- Part II: Approximate Solution Methods
  - Prediction and Control using Approximation
  - Eligibility Traces
  - Policy Gradient Methods
- Part III: Looking Deeper
  - **Deep Reinforcement Learning**
  - Partial Observability
  - Reward Engineering
  - Current Applications

Summary of Notation*

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\boldsymbol{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\boldsymbol{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bolt)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution $p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X, i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\text{argmax}_a f(a)$	a value of action a at which $f(a)$ takes its maximal value

Value Function

G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
Q, Q_t	array estimates of action-value function q_π or v_*

Temporal Difference Learning

U_t	target for estimate at time t
δ_t	TD error

*Some authors use a slightly different notation.

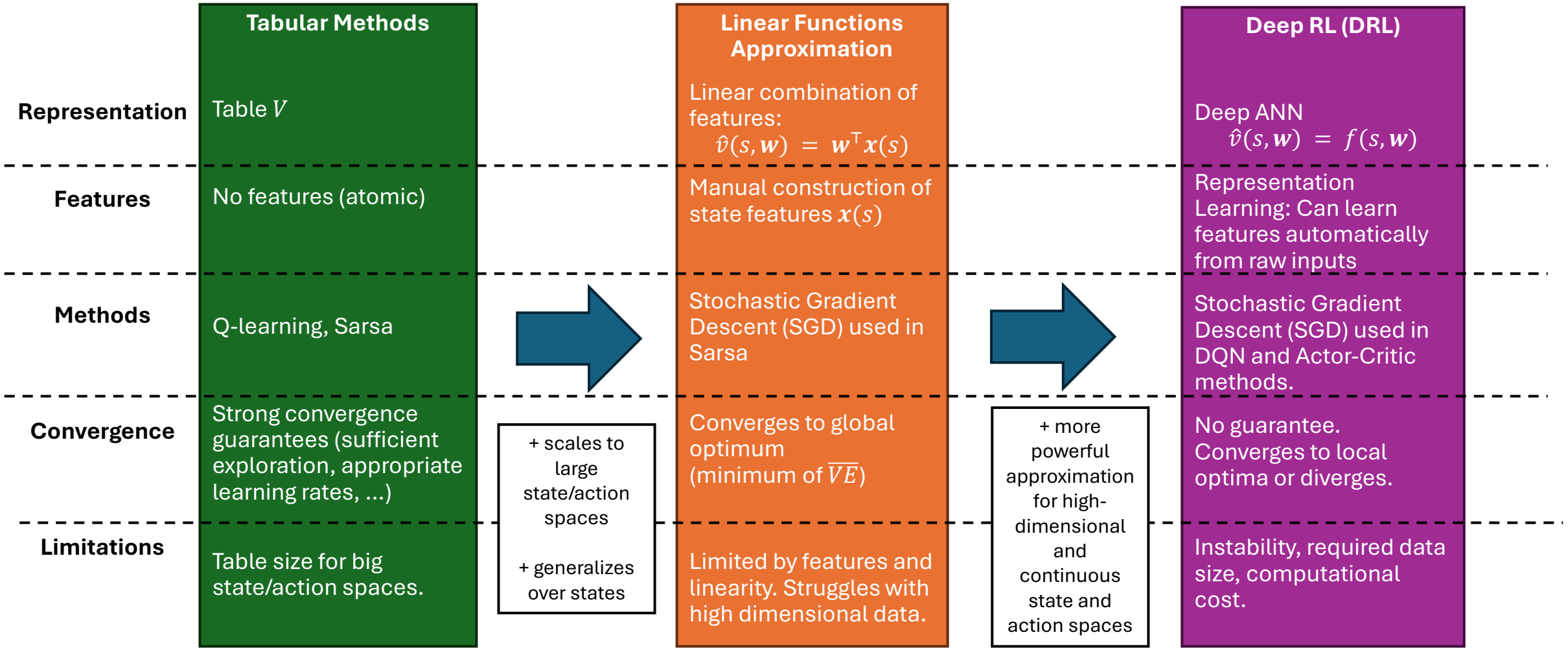
MDP

s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states, $\mathcal{S}^+$ are all states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r \mid s, a)$	probability of transition to state s' and receiving reward r, from state s taking action a.
$p(s' \mid s, a)$	probability of transition to state s' fom state s taking action a.
$r(s, a)$	expected immediate reward from state s after action a.
$r(s, a, s')$	expected immediate reward from state s to s' with action a.
$\pi(a \mid s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

Approximation

$\mu(s)$	on policy distribution (fraction of time spent in state)
α	step size (gradient descent learning rate)
$\hat{v}(s, \boldsymbol{w})$	approximate value function
$\hat{q}(s, a, \boldsymbol{w})$	approximate action-value function
$\pi(a \mid s, \boldsymbol{\theta})$	approximate policy
$\widehat{VE}$	mean squared value error
$\nabla f(x)$	gradient of function f evaluated at x

From Tabular RL to Deep RL



Disclaimer: Convergence

- So far, we have had strong convergence guarantees:
  - **Tabular RL** has the **γ-contraction Bellman operator**: guaranteed convergence to the optimal value function (a fixed point). The **policy improvement theorem** shows that the policy improves in every step.
  - **TD/Q-learning** converges with sufficient **exploration** and **decreasing learning rate**.
  - **Linear approximation:** Mean squared value error creates a **convex optimization problem**. On-policy learning converges to the global optimum.

- **Off-policy learning: Deadly Triade**

  Off-policy + bootstrapping + function approximation
  (two out of three are fine)

- **Non-linear approximation (DRL)** breaks most guarantees in RL!
  - **Loss of contraction mapping** due to a **non-linear projection**. A fixed point may not exist or not be reachable.
  - **Monotonic policy improvement guarantee is lost:** Policy can get worse.
  - **Oscillating or diverging targets:** Update targets needs to be fixed or converge at a slow rate (for TD). Non-linear function approximation often leads to oscillating or diverging targets.
  - **Off-policy learning** makes everything worse. See Deadly Triade.

Value-based DRL Methods

Use ANNs to represent a parameterized value function that is updated via gradient descent.

Remember: Value Functions

- Immediate reward: $r_t = r(s_t, a_t, s_{t+1})$

- Value functions:
  - $V^{\pi}(s) = \mathbb{E}_{\pi}[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid S_t = s\,, \pi]$
  - $Q^{\pi}(s,a) = \mathbb{E}_{\pi}[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid S_t = s, A_t = a\,, \pi]$

The expectation reflects all uncertainty:
- Stochastic policy
- Stochastic transitions
- Stochastic reward model

- Bellman equation:

$$Q^{\pi}(s,a) = \sum_{s' \in \mathcal{S}} p(s,a,s') \left(r(s,a,s') + \gamma Q^{\pi}\left(s', a = \pi(s')\right) \right)$$

- Optimality condition:

$$V^*(s) = \max_{\pi \in \Pi} V^{\pi}(s) \quad \text{and} \quad Q^*(s,a) = \max_{\pi \in \Pi} Q^{\pi}(s,a)$$

- Optimal policy: $\pi^*(s) = \operatorname{argmax}_{a \in \mathcal{A}} Q^*(s,a)$

Remember: Tabular Q-Learning

- An extremely influential method developed by Watkins (1989).
- Tabular update rule only uses Sars:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[\underbrace{R_{t+1} + \gamma \max_a Q(S_{t+1}, a)}_{\text{target}} - \underbrace{Q(S_t, A_t)}_{\text{prediction}}]$$

$$\text{TD-error}$$

- Q directly approximates q_* independent of the behavior policy being followed!
  → Off-policy
- Converges to the optimal value function if
  - State-action pairs are represented discretely (in a table).
  - All action are repeatedly sampled in all states (behavior policy keeps exploring).

- **Issue**: $Q(s, a)$ can often not be represented in a table due to too many action-value pairs. Use a parameterized approximation $Q(s, a; \boldsymbol{\theta})$. Deadly triade:
  - Bootstrapping
  - off-policy learning
  - function approximation
- Linear approximation converges for special cases. Non-linear approximation is an issue!

Fitted Q-Learning

Notation: DRL literature uses often Q where our textbook uses $\hat{q}$

- Use a parameterized approximation $Q(s, a; \boldsymbol{\theta})$
- Initial setting for $\boldsymbol{\theta}_0$ such that all Q-value are close to 0 (helps with learning).
- Bootstrapped target value at iteration k:

$$U_k = r + \gamma \max_{a'} Q(s', a'; \boldsymbol{\theta}_k)$$

Q-learning assumes the best action will be taken

- Update:

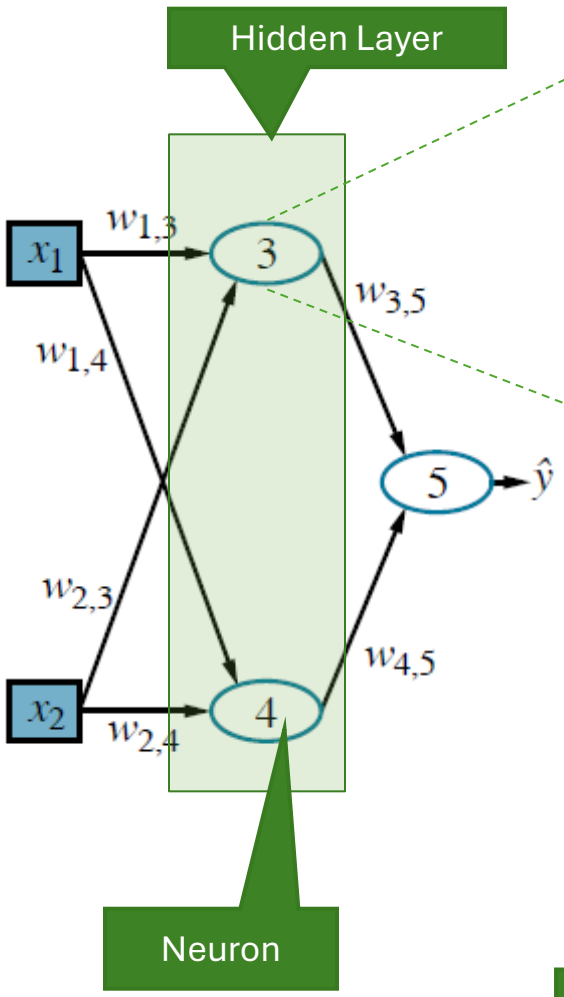
$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha\big(U_k - Q(s, a, \boldsymbol{\theta}_k)\big)\nabla Q(s, a, \boldsymbol{\theta}_k)$$

- Linear approximation with transformed state features can be used.
- Next, we will consider an artificial neural network (ANN) as the approximator.

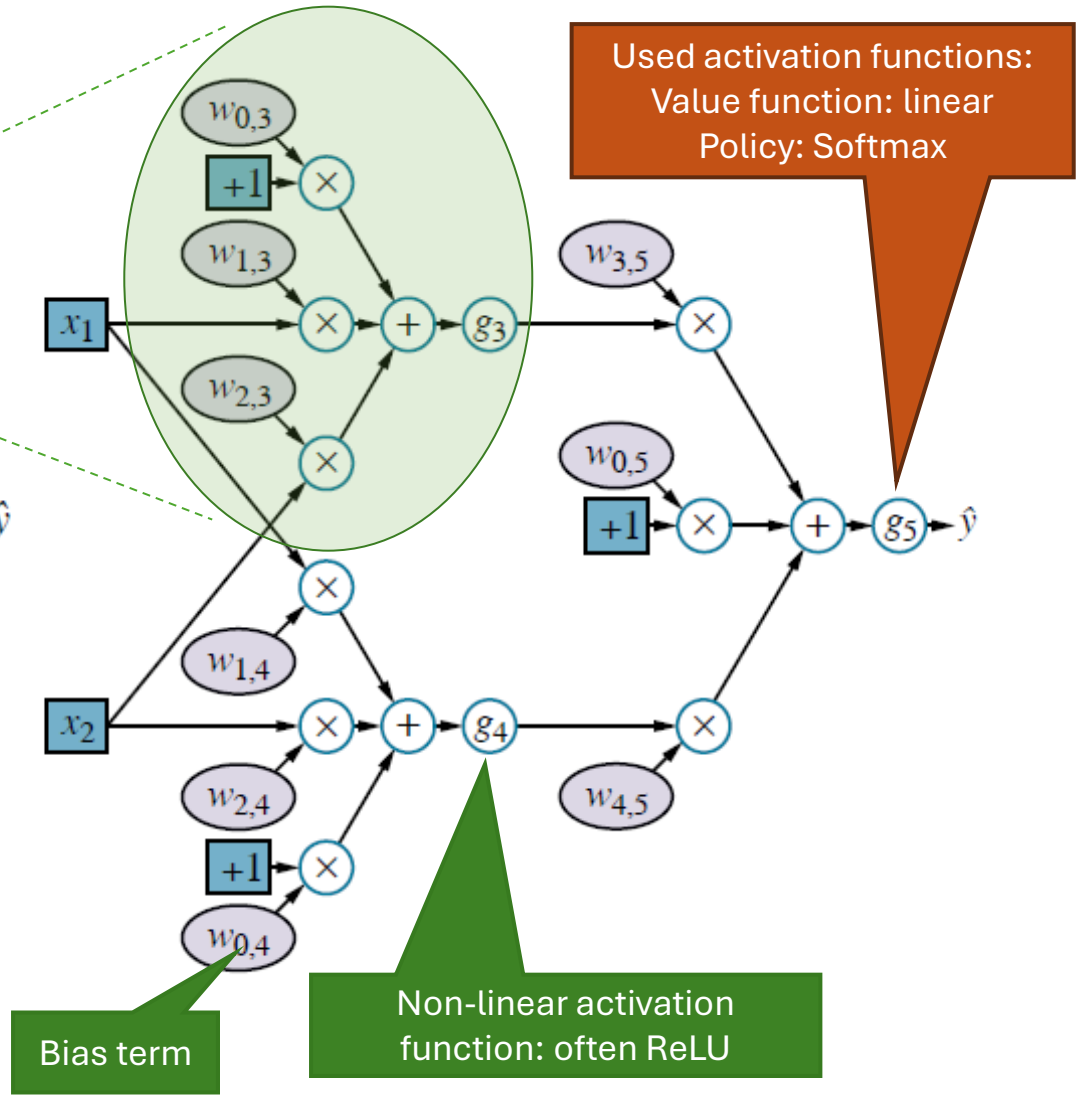
Reminder: Artificial Neural Networks

Superscript $[n]$ means the layer. Layer weights are collected in a matrix.

Network Topology



Computational graph

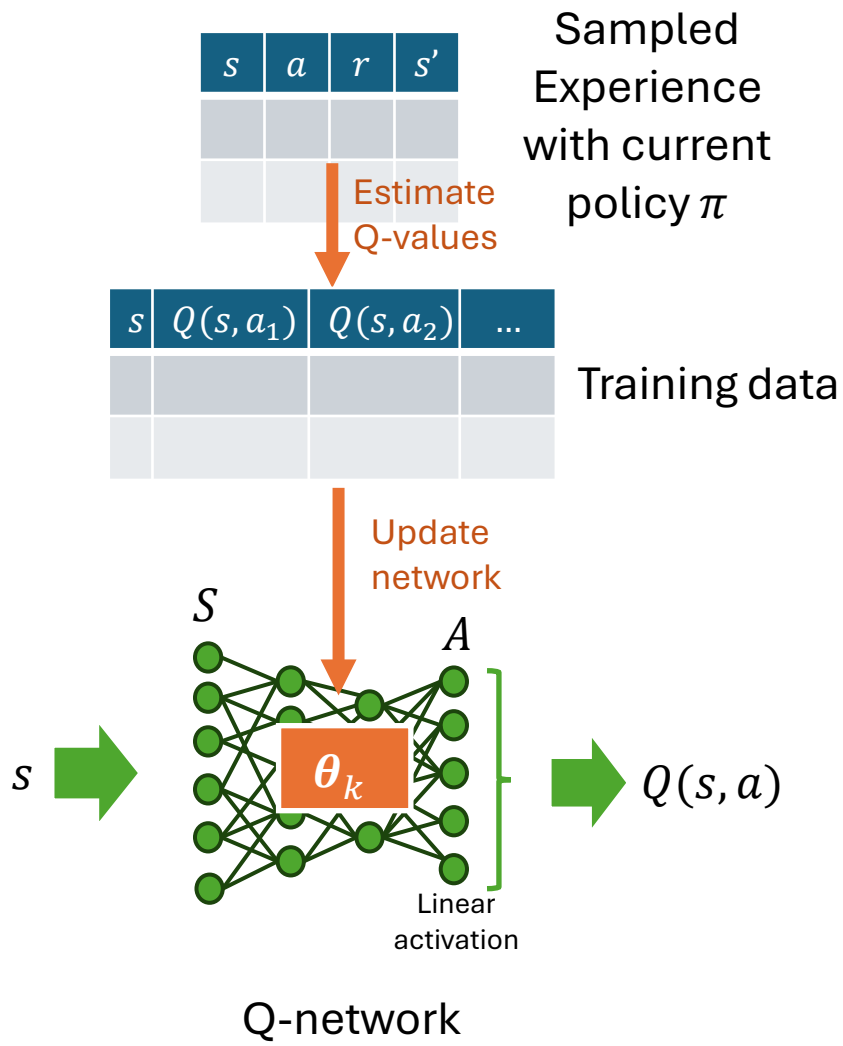


- Represent $\hat{y} = h(\boldsymbol{x}) = g^{[2]}\left(\boldsymbol{W}^{[2]}\, g^{[1]}\left(\boldsymbol{W}^{[1]}\boldsymbol{x}\right)\right)$ as a network of weighted sums with non-linear **activation functions** $g(\cdot)$ (e.g., sigmoid, ReLU).
- Learn weight matrices $\boldsymbol{W}$ from examples using gradient descent with **backpropagation** of prediction errors $L(\hat{y}, y)$.
- ANNs are **universal approximators**. Large networks can approximate any function (has no bias). **Regularization** is typically needed to avoid overfitting.
- **End-to-end learning**: The hidden layer performs "automatic feature engineering.
- **Deep learning** adds more hidden layers and layer types (e.g., convolution layers) for more efficient learning and transfer learning.

Notation: We use for the network weights $\boldsymbol{W}$ often $\boldsymbol{\theta}$

Neural fitted Q-learning (NFQ; Riedmiller, 2005)

- Use an ANN called the **Q-network** as the approximator for the Q-function.
- **Issue**: Online learning does not work well with our hardware. (GPUs only efficiently perform **many** matrix operations in parallel)
- **Solution**:
  - Learn the Q-network offline: **batch learning** with **experience replay** from a sample set of experience containing (s, a, r, s') tuples.
  - Needs off-policy learning: Use **supervised learning (regression)** with a sample from a replay buffer as the training set.
  - The regression target (Q-values) are estimated from the tuples using **temporal differencing**: $Q(s,a) = r + \gamma \max_{a'} Q(s', a')$
- **More issues**:
  - May not converge or become unstable (uses non-linear approximation where the target moves).
  - Q-values tend to be overestimated due to the max operator in the target.



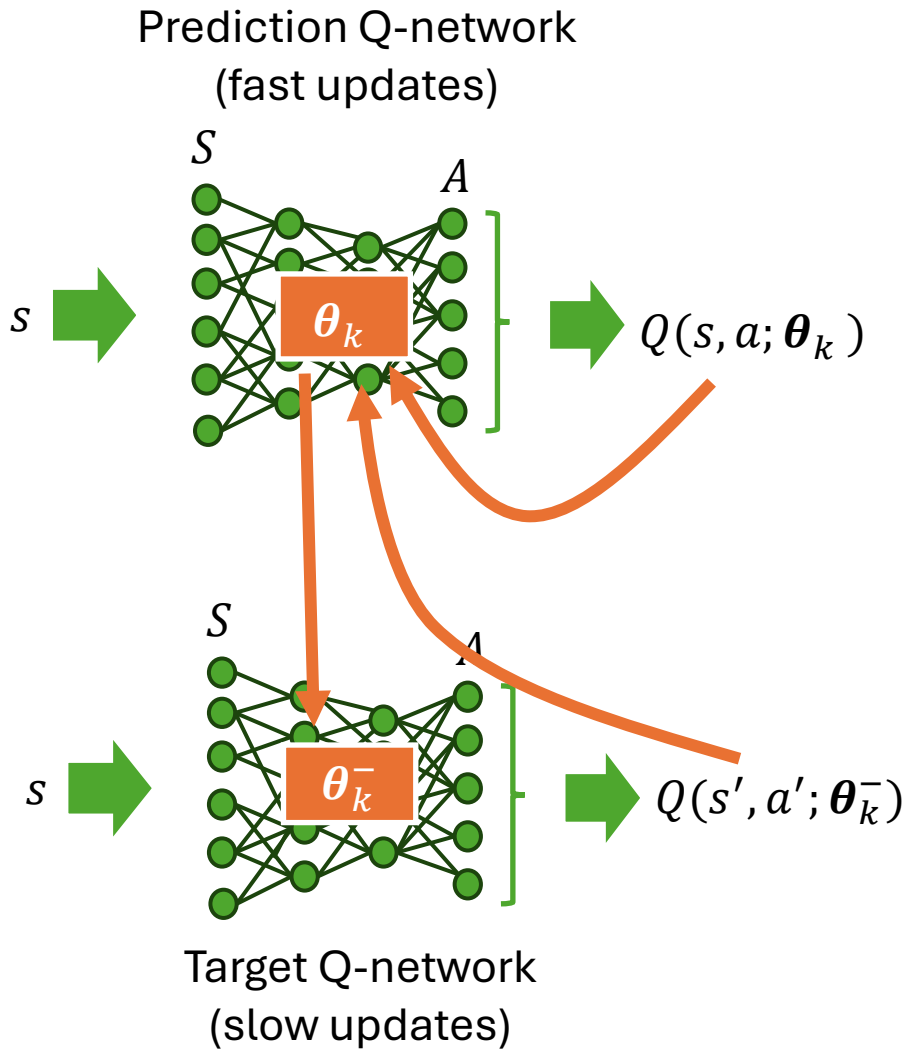
Deep Q-Network (DQN; Minh et al, 2015)

- DNQ addresses NFQ's **instability** of the target estimate.
- **Idea**: reduce instability by using a second target Q-network that is fixed (updated slowly):

$$U_k = r + \gamma \max_{a'} Q(s', a'; \boldsymbol{\theta}_k^-)$$

- **Updates**:
  - $\boldsymbol{\theta}_k$ is updated every iteration:
    $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha\big(U_k - Q(s, a, \boldsymbol{\theta}_k)\big)\nabla Q(s, a, \boldsymbol{\theta}_k)$
  - $\boldsymbol{\theta}_k^-$ is updated only every C iterations by $\boldsymbol{\theta}_k^- = \boldsymbol{\theta}_k$.

- **Effect**: Target U_k estimates are kept constant for C iterations, instabilities cannot propagate quickly.



Experience Replay

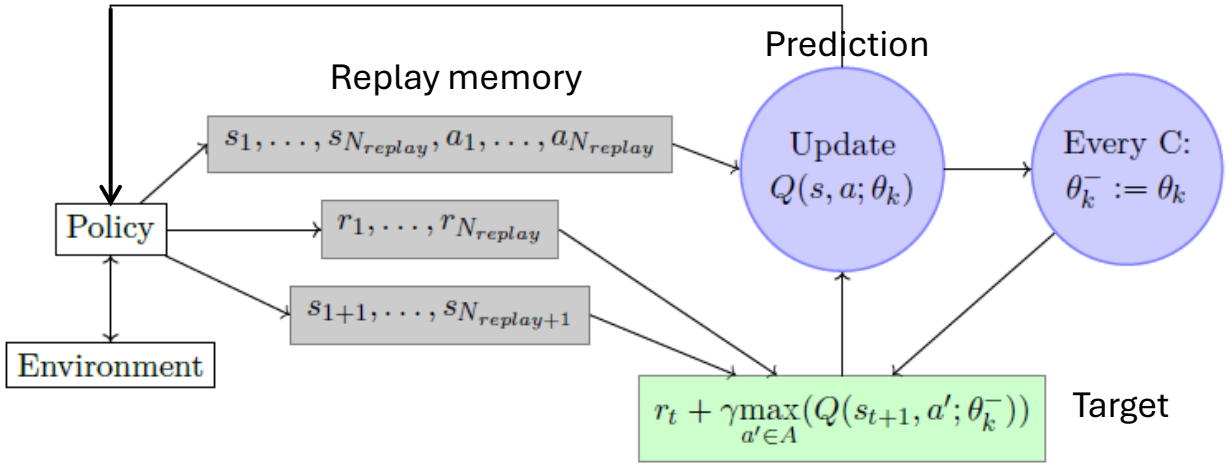
Issues

- Sample efficiency, GPU's process batches
- Training data should be i.i.d., but episodes are sequential.

Idea

- Reuse data (**replay memory**) and sample batches to destroy temporal correlation.

Restriction: Requires an off-policy learning algorithm!



Algorithm: DQN

1. Initialize $\boldsymbol{\theta} = \boldsymbol{\theta}^-$ so that all $Q(s, a; \boldsymbol{\theta})$ are close to 0.
2. Use an ϵ-greedy policy based on the current $\boldsymbol{\theta}$ to collect experience in the form (s, a, r, s') tuples and add them to the replay memory. If the memory is full, then old tuples will be overwritten.
3. Mini-batches (e.g., 32 elements) are taken randomly from the replay memory calculate gradients and to update $\boldsymbol{\theta}$.
4. $\boldsymbol{\theta}^-$ is set to $\boldsymbol{\theta}$ every C iterations.
5. Go back to step 2 till the maximum number of iterations is reached.

Code example:

DQN From Scratch - Lunar Lander

See Code on https://michael.hahsler.net/Introduction_to_Reinforcement_Learning/

Double DQN (DDQN; Van Hasselt, 2016)

- **Issue**: Q-learning and DQN use one function in the target to choose the action and evaluate its value. The max creates an **overestimation bias**.

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha \Big(\overbrace{r + \gamma \max_{a'} Q(s', a'; \boldsymbol{\theta}_k)}^{\text{target}} - \overbrace{Q(s, a, \boldsymbol{\theta}_k)}^{\text{prediction}} \Big) \nabla Q(s, a, \boldsymbol{\theta}_k)$$

$$\text{TD-error}$$

- DDQN **decoupling the action selection and evaluation** by using different parameters:

$$U_k = r + \gamma Q\left(s', \operatorname*{argmax}_a Q(s', a'; \boldsymbol{\theta}_k); \boldsymbol{\theta}_k^-\right)$$

Action evaluation uses more stable $\boldsymbol{\theta}_k^-$

Action selection uses $\boldsymbol{\theta}_k$

Advantage Function

- The “advantage” of choosing action a over the baseline given by the state value is

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

- Positive advantage means action a is better than the action that $\pi(s)$ chooses.

  For the optimal π^*:

$$A^{\pi^*}(s,a) = \begin{cases} 0 & \text{if } s = s^* \\ < 0 & \text{otherwise.} \end{cases}$$

- Advantage can be used in algorithms as a direct replacement for the Q-function.
- **Benefit**: Advantage typically produces a **stronger, lower variance signal** to improve the policy and speeds up learning.

Dueling Network Architecture (Wang, 2015)

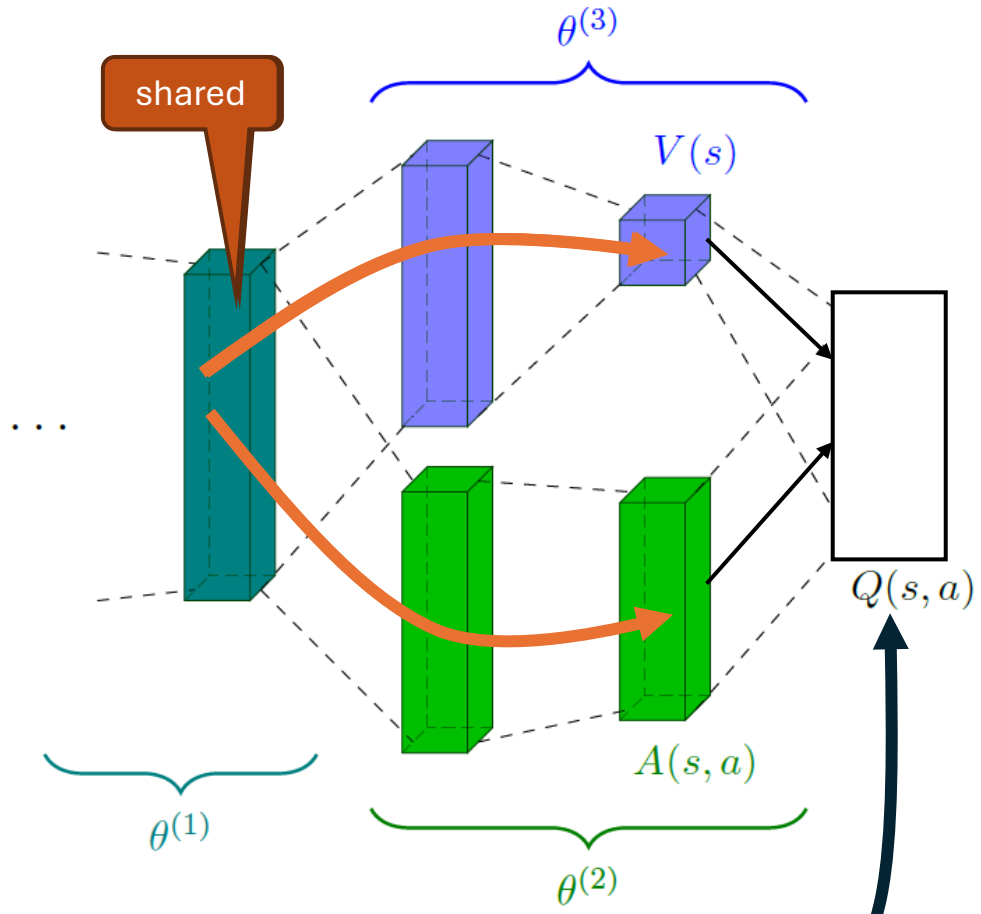
- Dueling Network Architecture only changes the ANN architecture. It learns two components:
  - Value network $V(s; \boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(3)})$
  - Advantage network $A(s, a'; \boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(3)})$
- $Q(s, a)$ is calculated using the equation below.

- Reminder: Advantage function

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

gives the "advantage" of choosing action a over the state value.

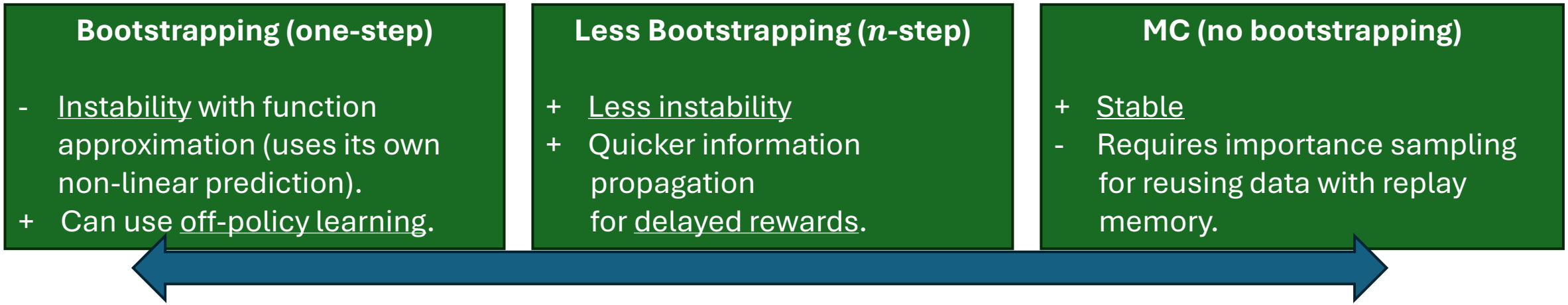
- This approach typically leads to improved performance.



$$Q(s, a; \boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}, \boldsymbol{\theta}^{(3)}) = V(s; \boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(3)}) + \left(\mathrm{A}(\mathrm{s}, \mathrm{a}; \boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}) - \max_{a' \in \mathcal{A}} \mathrm{A}(\mathrm{s}, \mathrm{a}'; \boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)})\right)$$

Becomes 0 for $a = a^*$

n-step Methods in DQN-type Algorithms



- Use n-step target for DQN-type algorithms:

$$U_k^n = \sum_{t=0}^{n-1} \gamma^t r_t + \gamma^n \max_{a' \in \mathcal{A}} Q(s_n, a'; \boldsymbol{\theta}_k)$$

- Eligibility Traces are difficult to use in DRL.

Code example:

DQN (Stable Baselines3) – L Maze

See Code on https://michael.hahsler.net/Introduction_to_Reinforcement_Learning/

Policy Gradient DRL Methods

Use ANNs to represent and learn a parameterized policy.

Remember: Policy Gradient Theorem

- We define the performance measure as the true value of following π_θ from the start state s_0

$$J(\boldsymbol{\theta}) \stackrel{\text{def}}{=} v_{\pi_\theta}(s_0) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s_0\right]$$

- The **policy gradient theorem** gives us an analytical expression for the unbiased gradient of the performance with respect to the policy parameters:

$$\nabla J(\boldsymbol{\theta}) = \mathbb{E}_\pi[q_\pi(s,a)\nabla \log \pi(a|s)] \propto \sum_s \mu(s) \sum_a q_\pi(s,a)\nabla\pi(a|s,\boldsymbol{\theta})$$

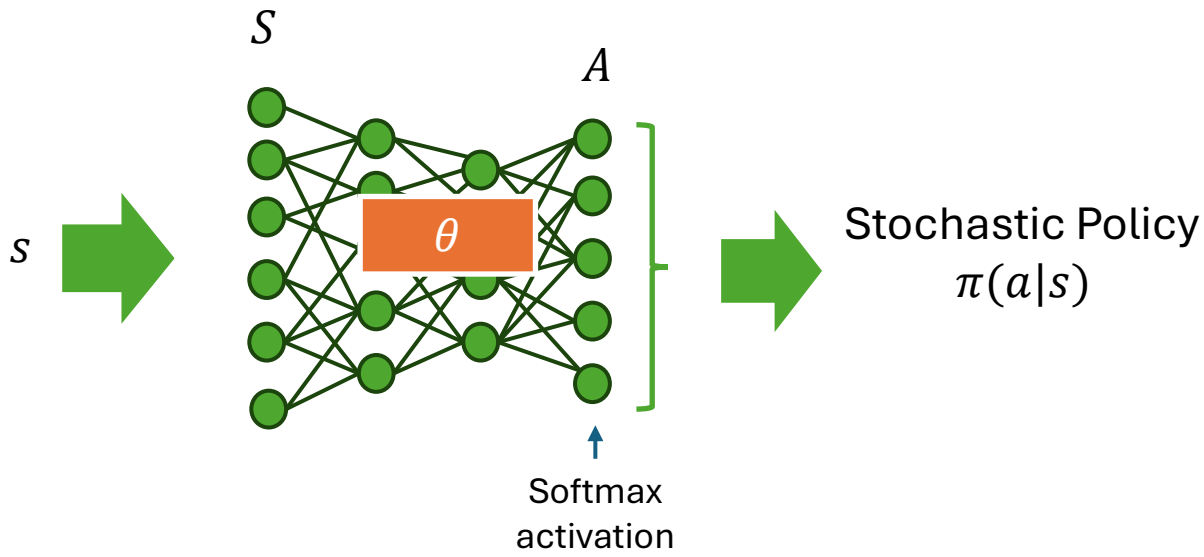
On-policy distribution under π

Depends on estimates for q

Gradient of the policy approximation function.

- **Convergence?**
  - Does **NOT** guarantee: convergence to (global) optimum, fast convergence or stability!
  - REINFORCE provides some guarantees of convergence to local optima **only for linear approximation**.

Policy Network



- Use an ANN to approximate a **stochastic policy**.
- Output layer uses **softmax activation** to produce the probability distribution for a stochastic policy.

- **Goal**: $J(\boldsymbol{\theta}) \stackrel{\text{def}}{=} v_{\pi_\theta}(s_0)$
- **Loss**: $L(\theta) = -\ln \pi(A_t|S_t, \theta)\, G$
- **Update**: $\theta \leftarrow \theta - \alpha \nabla_\theta L(\theta)$

Means that the highest probability action leads to the largest return. Can be reformulated as a loss.

REINFORCE With Policy Network

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot,\boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ (G_t)
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t,\boldsymbol{\theta})$

Represent the policy as a policy network.
This is equivalent to :

$$L(\theta) = -\ln \pi(A_t|S_t,\theta)\, G$$
$$\theta \leftarrow \theta - \alpha \nabla_\theta L(\theta)$$

Note: MC leads to instability!

Remember: The Policy Gradient Update Signal

- True policy gradient

$$\nabla J(\boldsymbol{\theta}) = \mathbb{E}_\pi[q_\pi(s,a)\,\nabla \log \pi(a|s)]$$

- The update can use any signal X

$$\nabla J(\boldsymbol{\theta}) \propto \mathbb{E}_\pi[X\,\nabla \log \pi(a|s)]$$

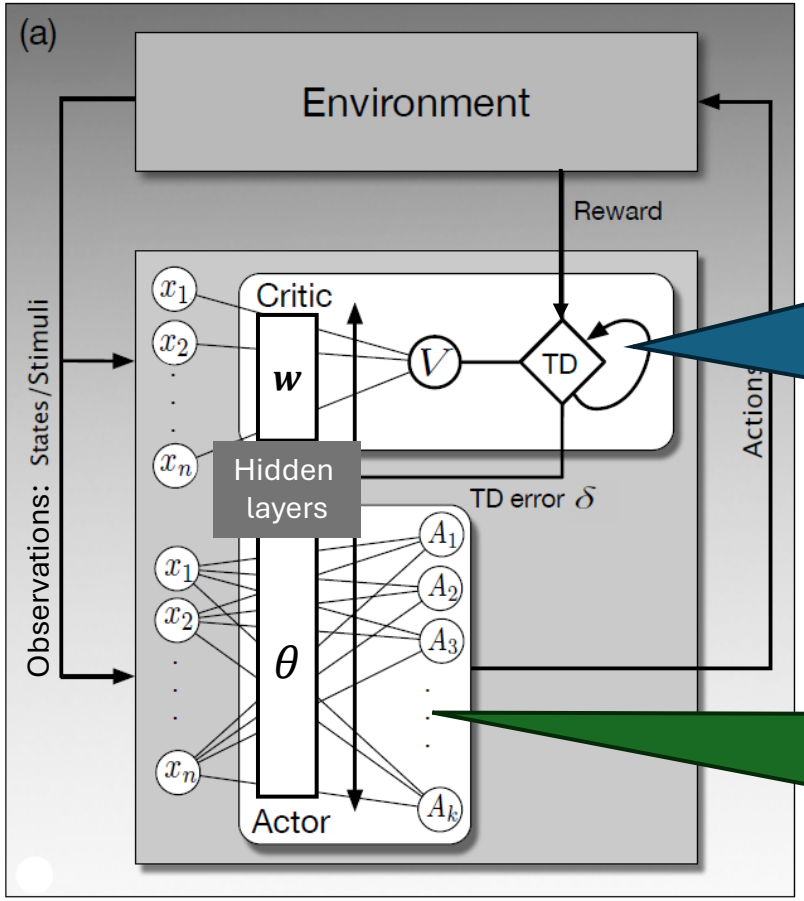
  as long as $\mathbb{E}_\pi[X|s,a] \propto q_\pi(s,a)$, i.e., if it tells us how good an action is.

- Examples of valid signals:
  - MC (REINFORCE): $X = G_t$
  - Advantage function: $X = A(s,a) = q(s,a) - v(s)$
  - TD error: $X = \delta_t = R_{t+1} + \gamma v(S_{t+1}) - v(S_t)$

Advantage: Measures how much better a is compared with the current action in $\pi(s)$.

Neural Actor-Critic Method

Use a value network and a policy network for **on-policy** control.



Critic

- **Value network** approximates the value function $\hat{v}(s, \boldsymbol{w})$
- Produces the TD error from the reward signal using its network's state-value predictions.
- Adjusts the state-value network using the TD error.
- Does not select actions!

Actor

- **Policy network** approximates the policy π_θ
- Selects actions.
- Adjusts the policy network based on the TD error.
- No direct access to reward signal!

Section 15.7 in Sutton and Barto (2018)

Neural Actor-Critic Method: One-Step Algorithm

One-step Actor–Critic (episodic), for estimating $\pi_{\boldsymbol{\theta}} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$

Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$

Parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^{d}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

  Initialize S (first state of episode)

  $I \leftarrow 1$

  Loop while S is not terminal (for each time step):

    $A \sim \pi(\cdot|S,\boldsymbol{\theta})$

    Take action A, observe S', R

    $\delta \leftarrow R + \gamma \hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})$ (if S' is terminal, then $\hat{v}(S',\mathbf{w}) \doteq 0$)

    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S,\mathbf{w})$

    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} I \delta \nabla \ln \pi(A|S,\boldsymbol{\theta})$

    $I \leftarrow \gamma I$

    $S \leftarrow S'$

actor

critic

Takes care of discounting γ^t

Policy approximation is soft, so it explores!

TD error: uses $\hat{v}$ estimates

Update the critic (value function) = $TD(0)$

Update the actor (policy) using the TD error signal

Replace these with a value network and a policy network.

Advantage Actor Critic: A2C

A popular synchronous version of Asynchronous Advantage Actor Critic (A3C).

Algorithm 7.2 Advantage Actor-Critic (A2C) Algorithm

1: input: Policy parameters θ, value function parameters w. Learning rates α_θ, α_w, discount factor γ and entropy coefficient β.
2: **for** each iterative step **do**
3: Collect trajectories with transitions $(s_t, a_t, r_t, s_{t+1}, d)$ for $t = 0, 1, \ldots, T$ with $a_t \sim \pi(s_t; \theta)$.
4: **for** each trajectory (or episode) **do**
5: Compute Advantage:

$$A(s_t, a_t) = r_{t+1} + \gamma V(s_{t+1}; w) * (1 - d) - V(s_t; w) \quad (7.12)$$

This is the 1-step estimate of $Q(s_t, a_t)$

▷ $A(s_t, a_t) \approx \delta_t$, TD error

$\mathbb{E}[\delta_t \mid s, a] = A(s, a)$

6: Update Critic:

$$w \leftarrow w + \alpha_w \nabla_w V(s_t; w) A(s, a)$$

▷ Critic minimises TD error

7: Update Actor:

$$\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \log \pi(a|s; \theta) A(s, a) + \beta H(\pi_\theta(s_t)) \quad (7.14)$$

▷ maximises actor

8: **end for**
9: **end for**

Note: d is a done indicator for the episode end

Advantage provides a better, lower variance signal by focusing on relative action quality. Could use multi-step bootstrapping for more stability and faster reward propagation.

Optional Entropy regularization: encourages the policy to stay uncertain/spread out and explore. (Not all implementations do this)

Source: https://medium.com/@kswagat1979/advantage-actor-critic-a2c-models-ab188ddfcb72

Natural Policy Gradients (NPG)

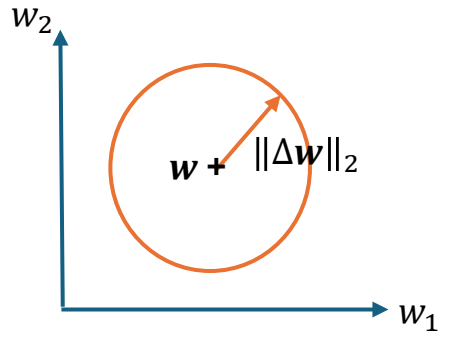
Notation: θ and w are often used interchangeably!

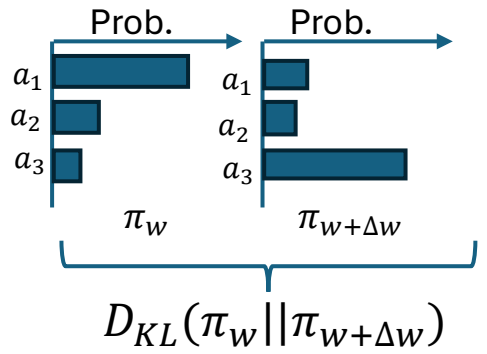
- Regular policy gradient methods
  - Update policy parameters $\boldsymbol{w}$ in the direction of the steepest ascent of the performance measure $J(\boldsymbol{w}) \stackrel{\text{def}}{=} v_{\pi_w}(s_0)$
  - The update changes the parameters $\Delta \boldsymbol{w} \propto \nabla_w J(\boldsymbol{w})$. The step size α restricts **moves in the parameter space** to some $\|\Delta \boldsymbol{w}\|_2$.
- **Issue**: The same amount of change in the parameter space $\boldsymbol{w}$ can have very different effects on the change of the stochastic policy π_w (the probability distribution over actions). This can cause **instability**.
- **Idea**: Natural policy gradients take the geometry of the policy (a probability manifold) into account.
  - **Measure of the difference between two policies**' action probability distributions**:** Kullback-Leibler (KL) divergence $D_{KL}(\pi_w || \pi_{w+\Delta w})$
  - Constraint $\Delta \boldsymbol{w}$ by a maximum allowable policy change using $D_{KL}(\pi_w || \pi_{w+\Delta w}) \leq \delta$.
- Local approximation: The Fisher information criterion can be used to calculate **the steepness of the gradient** for changes in the policy distribution. α now controls how much the policy changes:

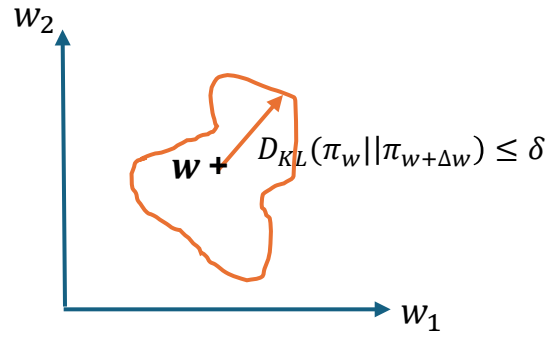
$$\Delta \boldsymbol{w} \propto F_w^{-1} \nabla_w J(\boldsymbol{w})$$

- **Problem**: Calculating the Fisher information matrix (F_w) is too expensive for deep learning models. We need an effective approximation.

  This idea inspired several state-of-the-art methods:
  - Trust Region Optimization (TRPO)
  - Proximal Policy optimization (PPO)







Trust Region Policy Optimization (TRPO; Schulman et al, 2015)

Notation: θ and w are often used interchangeably!

- An actor critic method using a simplification of the natural policy gradient.
- Surrogate objective. Instead of directly maximizing the value function, it uses the **probability ratio of choosing an action** before and after the update weighed by the advantage.
- It prefers updates that increases the probability of picking high-advantage actions.

$$\max_{\Delta w} \mathbb{E}_{\pi_w}\left[\frac{\pi_{w+\Delta w}(s,a)}{\pi_w(s,a)} A^{\pi_w}(s,a)\right]$$

Subject to: $\mathbb{E}[D_{KL}(\pi_w(s,\cdot)||\pi_{w+\Delta w}(s,\cdot))] \leq \delta$

- The constraint explicitly bounds the policy change per update step.
- δ is a tune-able hyperparameter specifying the size of the safe "trust region." This prevents destructive, large changes in the policy.

Proximal Policy Optimization (PPO; Schulman et al, 2017)

Notation: θ and w are often used interchangeably!

- A popular and more practical variant of TRPO.
- Also uses the probability ratio of choosing an action weighted by the advantage.
- Replaces the expensive KL-divergence constraint with a clipped surrogate objective function that keeps the policy changes within a probability ratio of $[1-\epsilon, 1+\epsilon]$

$$\max_{\Delta w} \mathbb{E}_{\pi_w}\left[\min\left(\frac{\pi_{w+\Delta w}(s,a)}{\pi_w(s,a)} A^{\pi_w}(s,a), \text{clip}_{1\pm\epsilon}\left(\frac{\pi_{w+\Delta w}(s,a)}{\pi_w(s,a)}\right) A^{\pi_w}(s,a)\right)\right]$$

- ϵ is a tunable hyperparameter.
- Typically achieves much of the stability of other NPG methods without the computational cost of calculating the KL divergence or the Fisher information matrix.

PPO Algorithm

Algorithm 1 PPO-Clip

1: Input: initial policy parameters θ_0, initial value function parameters ϕ_0
2: **for** $k = 0, 1, 2, \ldots$ **do**
3: Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
4: Compute rewards-to-go $\hat{R}_t$.
5: Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}.
6: Update the policy by maximizing the PPO-Clip objective:

Actor

$$\theta_{k+1} = \arg\max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

Advantage is more stable

typically via stochastic gradient ascent with Adam.

7: Fit value function by regression on mean-squared error:

Critic

$$\phi_{k+1} = \arg\min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2,$$

Clipping approximates the expensive **natural-gradient** computation.

typically via some gradient descent algorithm.

8: **end for**

Soft Actor-Critic (SAC)

- **Soft Actor-Critic (SAC)** is a popular **off-policy** method for **continuous action space.**

SAC uses a large set of enhancements that can also be used in other algorithms:

- **Maximum entropy objective:** Adds policy entropy regularization to encourage exploration:

$$\mathrm{J}(\theta) = \mathbb{E}\left[\sum_t r(S_t, A_t) + \alpha\mathcal{H}(\pi_\theta(\cdot\,|S_t))\right]$$

  The temperature α to balance exploration and exploitation is automatically tunes.

$\mathcal{H}$ is the policy's entropy measuring how spread out the policy probability distribution is.

- **Reparameterization trick** for actor policy: Instead of using $a \sim \pi_\theta(a|s)$, it samples from

$$a = f_\theta(\epsilon, s), \qquad \epsilon = \mathcal{N}(0,1)$$

  where a is a continuous action and f_θ is a learned ANN. This Gaussian actor is a popular choice since it has a closed-form solution for the entropy used in the objective and allows for backpropagation through actions.
- **Replay buffer:** Improve sample efficiency and updates can be done in batches on GPUs. Possible since it is off-policy.
- **Twin Q-networks**: The critic learns two Q-networks and used $\min(Q_1(s,a), Q_2(s,a))$ to reduce the maximization bias.
- **Slow update** of target network: $\theta_{\text{target}} \leftarrow \tau\theta + (1-\tau)\theta_{\text{target}}$

Idea from double Q-learning.

Instead of copying the network every C timesteps.

Code examples:

DQN and PPO (Stable Baselines3) - Lunar Lander

See Code on https://michael.hahsler.net/Introduction_to_Reinforcement_Learning/

Comparison of Popular DRL Methods

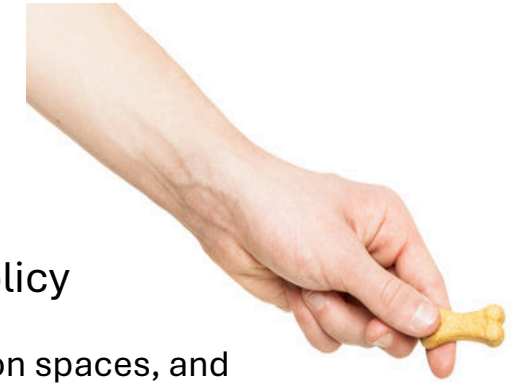
Method	Type	On/Off Policy	Action Space	Key Idea	Strengths	Weaknesses
DQN	Value-based	Off-policy	Discrete	Learn Q(s,a) with neural net	Simple, foundational	Unstable, overestimation bias
Double DQN	Value-based	Off-policy	Discrete	Reduces Q overestimation	More stable than DQN	Still limited to discrete
Dueling DQN	Value-based	Off-policy	Discrete	Separates value & advantage	Better learning efficiency	More complex
REINFORCE	Policy gradient	On-policy	Both	Monte Carlo policy gradient	Conceptually simple	High variance
TRPO	Policy gradient / Actor-critic	On-policy	Both	Constrained updates (KL bound)	Stable updates	Complex, slow
PPO ⭐	Actor-critic	On-policy	Both	Clipped objective for stability	Easy, robust, widely used	Less sample efficient
A2C	Actor-critic	On-policy	Both	Advantage-based updates	Simpler than PPO	Less stable
A3C	Actor-critic	On-policy	Both	Parallel workers	Faster training	Harder to implement
DDPG	Actor-critic	Off-policy	Continuous	Deterministic policy + Q-learning	Works for continuous control	Unstable, sensitive
TD3 ⭐	Actor-critic	Off-policy	Continuous	Twin critics + delayed updates	More stable than DDPG	Still sensitive
SAC ⭐	Actor-critic	Off-policy	Continuous	Max entropy RL + twin critics	Very stable, sample efficient	More computation

⭐ = most popular

Method Used in DRL

- **Policy Gradients** (REINFORCE and all actor-critic methods)
- **Natural Policy Gradients** (TRPO, PPO)
- **Actor–Critic Framework** (A2C, PPO, SAC, TD3, DDPG)
- **Stochastic policies** (PPO, SAC)
- **Advantage** estimation to reduce variance (PPO, A2C, TRPO)
- **Target Networks** to stabilize training (DQN, DDPG, TD3, SAC)
- **Experience Replay** for off-policy methods to improve sample efficiency and use hardware efficiently (DQN, DDPG, TD3, SAC)
- **Entropy Regularization** to prevent premature convergence of policy (SAC and sometimes in PPO)
- New approaches use **model learning** to learn environment dynamics (p) (Dreamer, MuZero)

What you Should Know



- DRL uses neural networks for non-linear value function or policy approximation, resulting in:
  - more powerful for high-dimensional and continuous state and action spaces, and
  - does not need explicit state feature engineering (representation learning).
  - However, most convergence guarantees are gone and instability is common.



- Issues:
  - Non-linear approximation means that small errors can get amplified through bootstrapping
  - The network chases a moving target.
  - Training can diverge or oscillate wildly.
  - Deadly Triad with off-policy learning
  - Sample efficiency: On-policy methods cannot use experience replay.

- Many tricks are used
  - DQN uses slower-moving target networks → reduce instability due to moving targets
  - Double DQN → decouple next action selection and q-value estimation to reduce overestimation bias
  - Experience replay → decorrelate data, improve sample efficiency, requires off-policy learning
  - Advantage normalization → reduce variance
  - Natural policy gradients → improve stability